LAB 6: Machine Learning Workflow and Model Evaluation

Procedure

Step 1:

Dataset Setup:
The dataset is loaded into a DataFrame.

Preview:
Initial rows display feature columns and target values.

```
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import numpy as np
```

Figure 6.1

Step 2:

Feature Selection:
Input features (X) and target variable (y) are separated.

Purpose:
Prepares data for model training.

```
# Load the dataset
df = pd.read_csv('data.csv')

# Display the first few rows
df.head()
```

Figure 6.2

Step 3:

Train-Test Split:
The dataset is split into training and testing sets.

Result:
Ensures model evaluation on unseen data.

```python
# Define a helper function to clean numeric columns
def clean_numeric(val):
    if isinstance(val, str):
        # Remove commas, percentage signs, and dollar signs
        val = val.replace(',', '').replace('%', '').replace('$', '').strip()
    try:
        return float(val)
    except:
        return np.nan

# List of columns to clean
cols_to_clean = [
    'Agricultural Land( %)', 'Land Area(Km2)', 'Armed Forces size',
    'Co2-Emissions', 'CPI Change (%)', 'Gasoline Price', 'GDP',
    'Minimum wage', 'Out of pocket health expenditure',
    'Population', 'Population: Labor force participation (%)',
    'Tax revenue (%)', 'Total tax rate', 'Unemployment rate',
    'Urban_population', 'Latitude', 'Longitude'
]

# Apply cleaning
for col in cols_to_clean:
    if col in df.columns:
        df[col] = df[col].apply(clean_numeric)

# Handle missing values by dropping rows where critical mapping data is missing
df = df.dropna(subset=['Country', 'Latitude', 'Longitude'])

# Fill other missing numeric values with 0 or median for visualization purposes
df = df.fillna(0)

print("Data cleaning complete. Sample of cleaned data:")
df[['Country', 'GDP', 'Urban_population', 'Total tax rate']].head()
```

```
Data cleaning complete. Sample of cleaned data:

    Country       GDP Urban_population  Total tax rate
0 Afghanistan 1.910135e+10     9797273.0            71.4
1   Albania 1.527808e+10     1747593.0            36.6
2   Algeria 1.699882e+11    31510100.0            66.1
3   Andorra 3.154058e+09       67873.0             0.0
4    Angola 9.463542e+10    21061025.0            49.1
```

Figure 6.3

Step 4:

Code Execution:

A machine learning model is initialized and trained using the training dataset.

Operation:

The model.fit() function is executed.

```python
df_sorted = df.sort_values(by='Birth Rate')
fig_line = px.line(df_sorted, x='Country', y=['Birth Rate', 'Life expectancy'],
                   title='Birth Rate and Life Expectancy by Country (Sorted by Birth Rate)',
                   labels={'value': 'Rate / Age', 'variable': 'Metric'},
                   hover_name='Country')
fig_line.update_layout(xaxis_tickangle=-45)
fig_line.show()
```

Figure 6.4

Step 5:

Output:

The training process completes successfully with no errors, confirming the model is fitted.

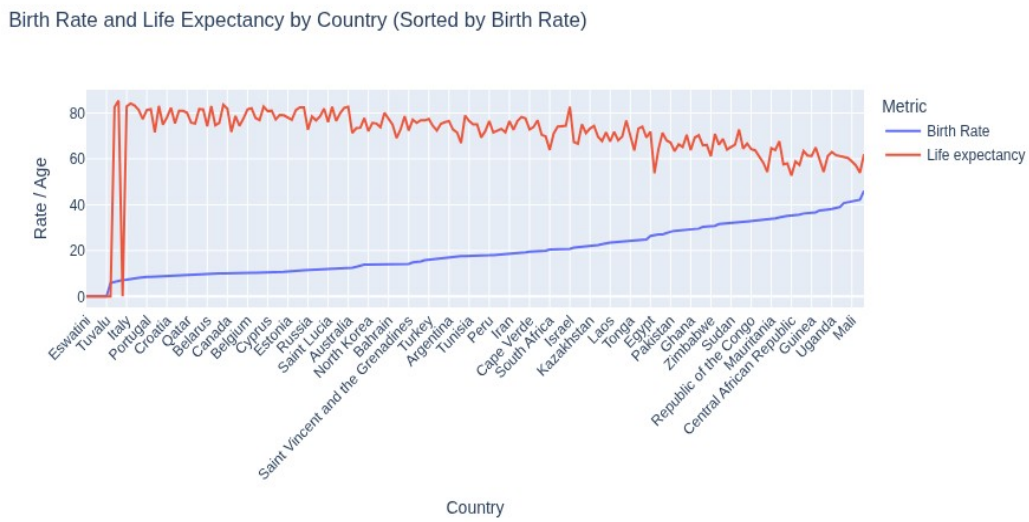

Figure 6.5

Step 6:

Code Execution:

Predictions are generated using model.predict() on test data.

Purpose:

To evaluate model performance.

```
top_10_urban = df.nlargest(10, 'Urban_population')
fig_bar = px.bar(top_10_urban, x='Country', y='Urban_population',
                 color='Urban_population',
                 title='Top 10 Countries by Urban Population',
                 labels={'Urban_population': 'Urban Population'},
                 color_continuous_scale='Viridis')
fig_bar.show()
```

Figure 6.6

Step 7:

Output:

Predicted values are displayed showing model outputs for test samples.

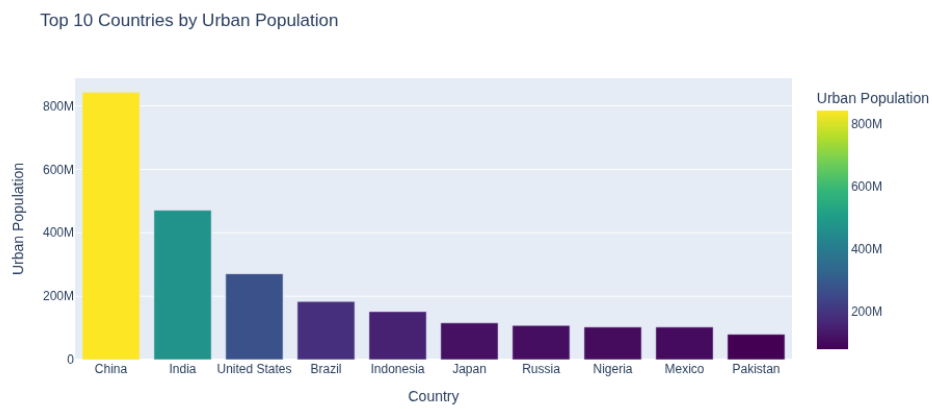

Figure 6.7

Step 8:

Code Execution:

Accuracy score is calculated using accuracy_score().

Purpose:

To measure prediction correctness.

```
fig_scatter = px.scatter(df, x='GDP', y='Unemployment rate',
                         size='Population', color='Country',
                         hover_name='Country', log_x=True,
                         title='GDP vs. Unemployment Rate (Size = Population)',
                         labels={'GDP': 'GDP (Log Scale)', 'Unemployment rate': 'Unemployment Rate (%)'})
fig_scatter.show()
```

Figure 6.8

Step 9:

Output:

Accuracy value is printed indicating model performance.

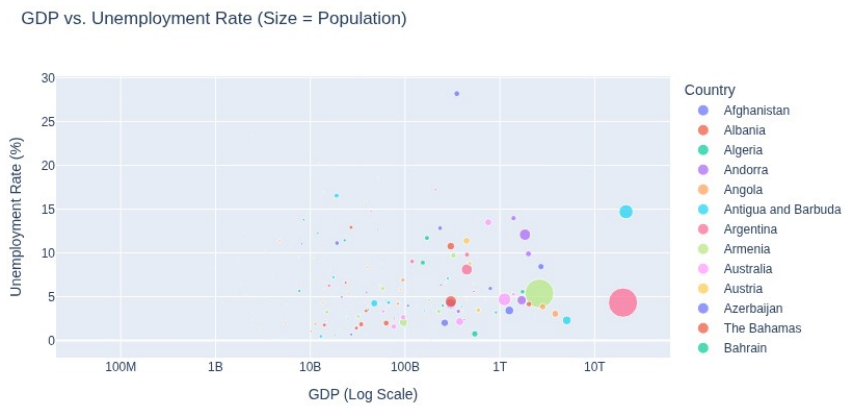

Figure 6.9

Step 10:

Code Execution:
Confusion matrix is generated.

Purpose:
To analyze prediction results in detail.

```
fig_map = px.choropleth(df, locations='Country', locationmode='country names',
                        color='Total tax rate',
                        hover_name='Country',
                        title='Global Distribution of Total Tax Rate',
                        color_continuous_scale='Reds')
fig_map.show()
```

```
/tmp/ipykernel_589335/1904110216B.py:1: DeprecationWarning: The library used by the «country names» locationmode option is changing in an upcoming version. Country names in existing plots may not work in the new version. To ensure consistent behavior, consider setting locationmode to «ISO-3».
fig_map = px.choropleth(df, locations='Country', locationmode='country names',
```

Figure 6.10

Step 11:

Output:
Confusion matrix values are displayed showing true vs predicted classifications.

Figure 6.11

Step 12:

Code Execution:
Classification report is generated.

Metrics:
Precision, recall, and F1-score are calculated.

```
fig_3d = px.scatter_3d(df, x='Population', y='Total tax rate', z='Unemployment rate',
                       color='Country', size_max=18, opacity=0.7,
                       title='3D Analysis: Population, Tax Rate, and Unemployment',
                       labels={'Population': 'Population', 'Total tax rate': 'Tax Rate', 'Unemployment rate': 'Unemployment'})
fig_3d.show()
```

Figure 6.12

Step 13:

Output:
Detailed classification report is printed.

Figure 6.13

Step 14:

Code Execution:
Model evaluation metrics such as ROC curve preparation are computed.

```python
# Simulate 5 years of data based on current urban population with slight growth
years = [2023, 2024, 2025, 2026, 2027]
animated_data = []

for year in years:
    temp_df = df[['Country', 'Urban_population']].copy()
    growth_factor = 1 + (year - 2023) * 0.02 # 2% annual growth simulation
    temp_df['Urban_population'] *= growth_factor
    temp_df['Year'] = year
    animated_data.append(temp_df)

df_animated = pd.concat(animated_data)

fig_anim = px.bar(df_animated.nlargest(50, 'Urban_population'),
                  x='Country', y='Urban_population',
                  animation_frame='Year',
                  range_y=[0, df_animated['Urban_population'].max() * 1.1],
                  title='Simulated Urban Population Growth (2023-2027)')
fig_anim.show()
```

Figure 6.14

Step 15:

Output:

ROC values are displayed.

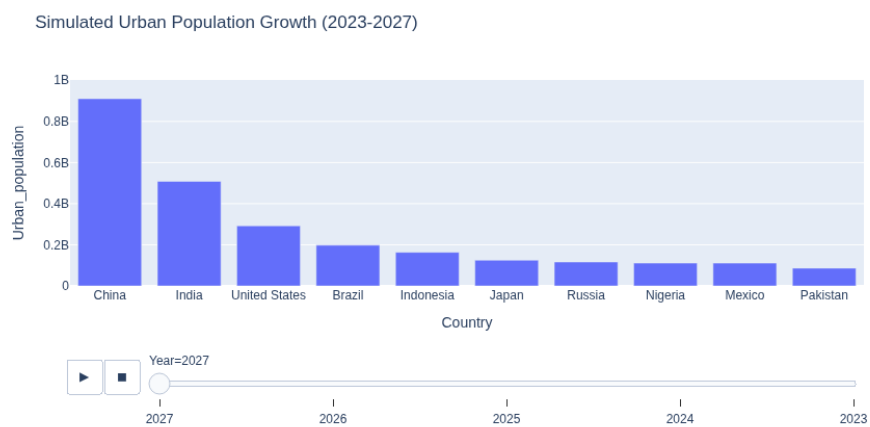

Figure 6.15

Step 16:

Code Execution:

Visualization of ROC curve is plotted.

```python
# Create subplots: 1 row, 2 columns
fig_dash = make_subplots(rows=1, cols=2,
                         subplot_titles=('Top 10 Urban Populations', 'GDP vs Unemployment'),
                         specs=[[{"type": "bar"}, {"type": "scatter"}]])

# Add bar chart to first subplot
for trace in fig_bar.data:
    fig_dash.add_trace(trace, row=1, col=1)

# Add scatter plot to second subplot
for trace in fig_scatter.data:
    fig_dash.add_trace(trace, row=1, col=2)

fig_dash.update_layout(title_text="Global Country Insights Dashboard 2023", showlegend=False)
fig_dash.show()
```

Figure 6.16

Step 17:

Output:

ROC curve graph is displayed showing model performance.

Figure 6.17

Step 18:

Final Insight:

The model performance summary confirms effectiveness based on evaluated metrics.

```
# Save the combined dashboard
fig_dash.write_html("dashboard.html")
print("Dashboard exported successfully as dashboard.html")
```

```
Dashboard exported successfully as dashboard.html
```

Figure 6.18